

# BloodChain

## Blockchain-Based Blood Bank System

Lahari Chowdary Taslila

Code Repository: <https://github.com/chowdary03/blood-bank>

### 1. Project Overview

**BloodChain** is a decentralized application (dApp) that digitizes the blood banking lifecycle on Ethereum. It introduces transparency, donor incentives, real-time expiry tracking, and a trustless peer-to-peer marketplace, replacing centralized, opaque blood-inventory systems.

| Traditional Pain Point                    | BloodChain Solution                                                                    |
|-------------------------------------------|----------------------------------------------------------------------------------------|
| Centralized registries prone to tampering | Immutable on-chain records; off-chain metadata anchored by <code>keccak256</code> hash |
| No donor incentive                        | YODA token appreciation reward paid on every altruistic donation                       |
| Expired blood goes unnoticed              | On-chain expiry check + public bounty for flagging stale units                         |
| No transparent pricing                    | Open marketplace with fee logic enforced in contract code                              |
| No traceability                           | Full lifecycle: Available → Reserved → Sold → Transfused / Expired                     |

#### 1.1 Deployed Contracts (Sepolia Testnet)

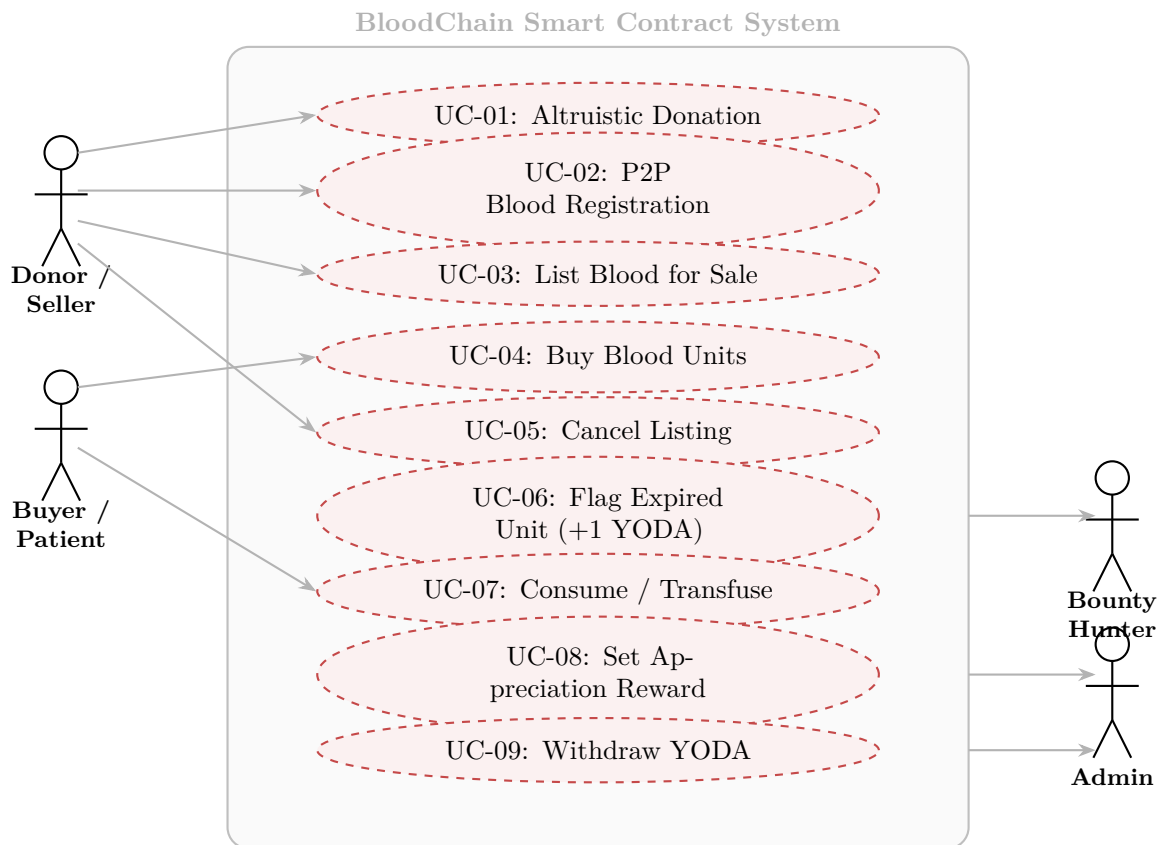
| Contract             | Address                                    |
|----------------------|--------------------------------------------|
| YodaToken (external) | 0xbd27d0b7F9fedb5A2A2C3ceF5dC9c70f3CF64Af2 |
| BloodRegistry        | 0xE4Fadfe917B8Dd89795C018796bcca26263FE712 |
| BloodMarket          | 0xbd800696238767c9b3ef26A95B4018D0b876eaD  |
| BloodDonation        | 0xa8e6b2ffD19359EE63a693e0D712b53E194385D0 |

### 2. Use Cases

#### 2.1 Actors

| Actor           | Role                                                         |
|-----------------|--------------------------------------------------------------|
| Donor / Seller  | Donates blood altruistically or registers units for P2P sale |
| Buyer / Patient | Purchases blood tokens using YODA on the marketplace         |
| Admin           | Deploys contracts, manages donated stock, collects P2P fees  |
| Bounty Hunter   | Any address that flags an expired unit and earns 1 YODA      |
| Facility        | Burns (consumes) blood tokens after transfusion              |

## 2.2 Use Case Diagram



## 2.3 Donation Process Flow

```

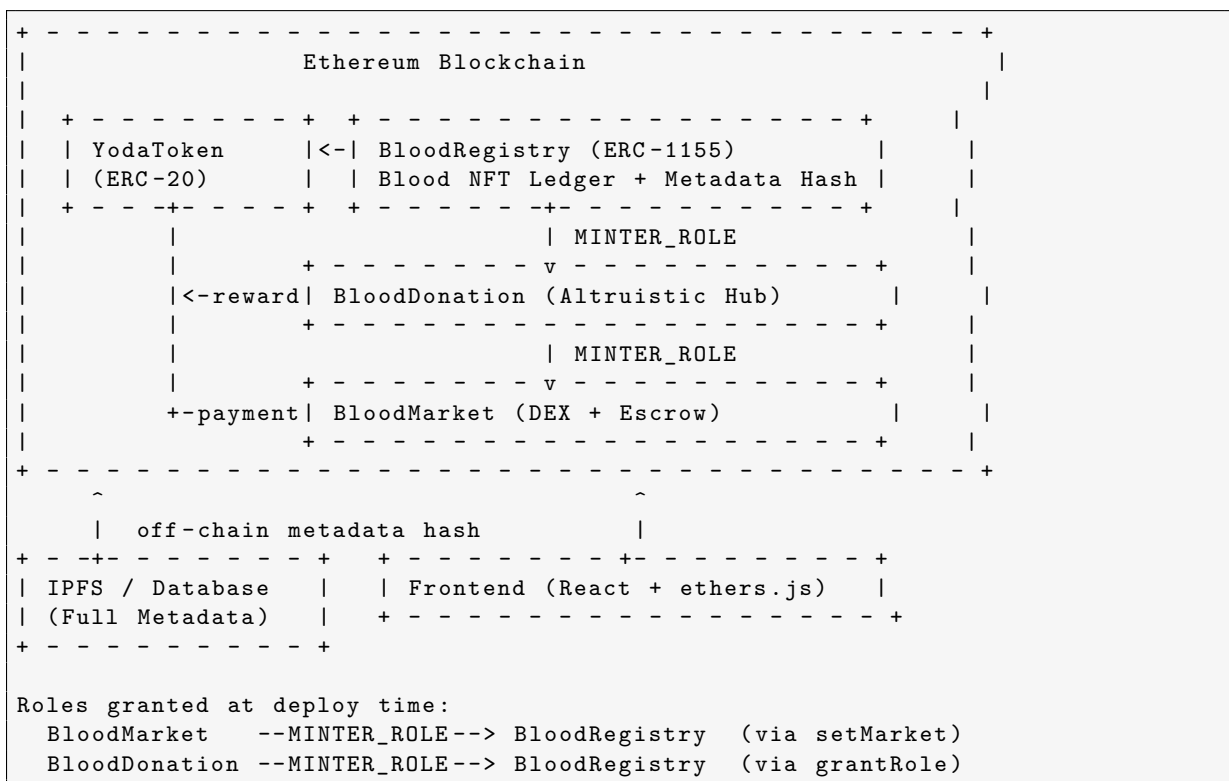
Donor --> donate(bloodGroup, rhPositive, component,
                expiryTime, metadataHash, amount)
|
+--> registry.registerUnit(
|     recipient    = owner(),          // tokens go to ADMIN
|     actualDonor = msg.sender         // donor identity preserved
| ) --> returns tokenId
|
+--> if (yoda.balanceOf(this) >= appreciationReward):
|     yoda.safeTransfer(msg.sender, appreciationReward)
|
+--> emit Donated(tokenId, msg.sender, owner(), amount, rewardPaid)
  
```

## 2.4 Use Case Summary Table

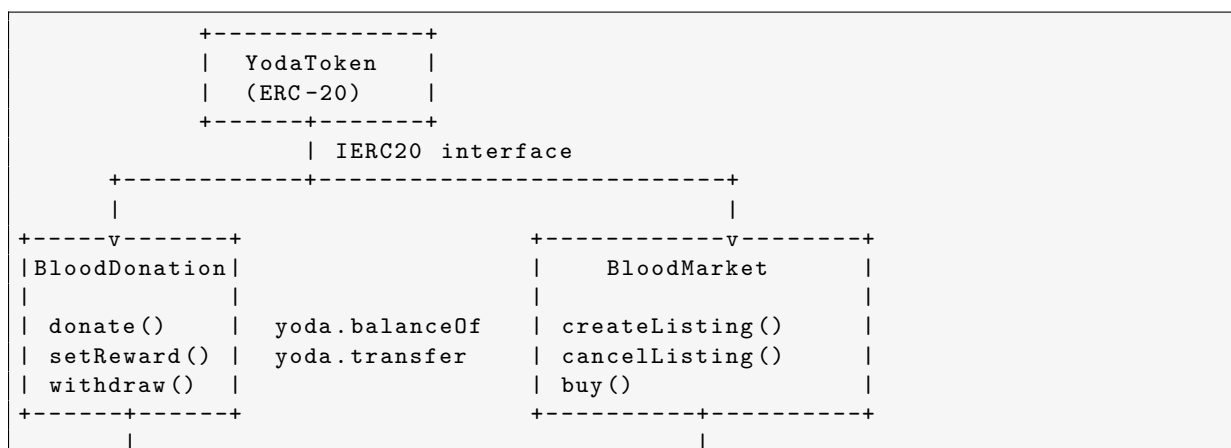
| # | Use Case            | Actor        | Contract / Function          | Outcome                             |
|---|---------------------|--------------|------------------------------|-------------------------------------|
| 1 | Altruistic Donation | Donor        | BloodDonation.donate()       | Tokens → Admin;<br>YODA → Donor     |
| 2 | P2P Registration    | Seller       | BloodRegistry.registerUnit() | Tokens minted to seller             |
| 3 | List for Sale       | Seller/Admin | BloodMarket.createListing()  | Tokens escrowed; listing active     |
| 4 | Buy Blood           | Buyer        | BloodMarket.buy()            | YODA transferred;<br>tokens → Buyer |
| 5 | Cancel Listing      | Seller       | BloodMarket.cancelListing()  | Tokens returned to seller           |

| # | Use Case            | Actor    | Contract / Function   | Outcome                         |
|---|---------------------|----------|-----------------------|---------------------------------|
| 6 | Flag Expired Unit   | Anyone   | BloodRegistry.flagExp | Status = Expired; 1 YODA bounty |
| 7 | Consume / Transfuse | Facility | BloodRegistry.consume | Tokens burned; lifecycle ends   |
| 8 | Set Reward          | Admin    | BloodDonation.setRew  | Donation reward updated         |
| 9 | Withdraw YODA       | Admin    | BloodDonation.withdr  | Unused YODA to admin            |

### 3. System Architecture



### 4. Contract Interaction Diagram



|                               |                          |                                  |                   |
|-------------------------------|--------------------------|----------------------------------|-------------------|
|                               | calls registerUnit()     |                                  | reads / transfers |
|                               | (recipient = admin)      |                                  | registry tokens   |
|                               |                          |                                  |                   |
| v                             |                          | v                                |                   |
| +-----+-----+-----+-----+     |                          |                                  |                   |
|                               | BloodRegistry (ERC-1155) |                                  |                   |
|                               |                          |                                  |                   |
|                               | registerUnit()           | updateStatus()                   | setMarket()       |
|                               | consumeUnit()            | isExpired() [view]               |                   |
|                               | flagExpired()            | --> pays 1 YODA bounty to caller |                   |
| +-----+-----+-----+-----+     |                          |                                  |                   |
| Roles granted at deploy time: |                          |                                  |                   |
| BloodMarket                   | --MINTER_ROLE-->         | BloodRegistry                    | (via setMarket)   |
| BloodDonation                 | --MINTER_ROLE-->         | BloodRegistry                    | (via grantRole)   |

## 5. Smart Contract Design Principles

### 5.1 Separation of Concerns

| Contract      | Standard                      | Single Responsibility                                |
|---------------|-------------------------------|------------------------------------------------------|
| YodaToken     | ERC-20                        | Settlement currency for all payments and rewards     |
| BloodRegistry | ERC-1155                      | Mint, track, and retire semi-fungible blood tokens   |
| BloodDonation | Ownable                       | Accept altruistic donations; distribute YODA rewards |
| BloodMarket   | Ownable +<br>Reentrancy-Guard | Escrow-based listing, buying, fee collection         |

### 5.2 Minimal On-Chain Storage

Only the fields that contract logic *enforces* live on-chain in the gas-optimised **BloodInfo** struct: blood group, Rh factor, component, status, timestamps, donor address, and a **keccak256** hash of the full off-chain JSON. All heavy metadata (volume, temperature, priority) is stored off-chain; the hash ensures any tampering is cryptographically detectable.

### 5.3 Role-Based Access Control (RBAC)

| Role               | Held By                       | Permission                                            |
|--------------------|-------------------------------|-------------------------------------------------------|
| DEFAULT_ADMIN_ROLE | Deployer                      | Grant/revoke roles; link market contract              |
| MINTER_ROLE        | BloodMarket,<br>BloodDonation | Mint tokens to addresses other than <b>msg.sender</b> |

Any user can self-mint (register their own blood) without a special role, enabling the P2P path. Only privileged contracts may mint to a *different* address.

### 5.4 Escrow Pattern

When a seller creates a listing, blood tokens are physically transferred *into* **BloodMarket**. This prevents double-listing without any off-chain coordination. Tokens release to the buyer on purchase, or back to the seller on cancellation.

## 5.5 Dual Payment Path

|                  |                                                      |
|------------------|------------------------------------------------------|
| Platform Listing | 100% → <i>admin</i> (donated blood sold by platform) |
| P2P Listing      | 95% → <i>seller</i> and 5% → <i>admin</i>            |

## 5.6 Incentive Mechanisms

1. **Donor Reward:** **BloodDonation** pays a configurable YODA amount to every altruistic donor.
2. **Expiry Bounty:** Anyone calling `flagExpired()` on a stale unit earns 1 YODA, keeping the on-chain ledger accurate.

## 6. Smart Contract Specifications

### 6.1 YodaToken (ERC-20)

ERC-20 token with **2 decimal places**. Sole payment currency. Uses an externally issued token on Sepolia; `YodaToken.sol` in the repo is a local reference for testing.

| Function                                | Access     | Description                         |
|-----------------------------------------|------------|-------------------------------------|
| <code>constructor(initialSupply)</code> | —          | Mints initial supply to deployer    |
| <code>mint(to, amount)</code>           | Owner only | Mint additional YODA to any address |
| <code>transfer / approve /</code>       | ERC-20 std | Standard token operations           |
| ...                                     |            |                                     |

### 6.2 BloodRegistry (ERC-1155)

Central ledger. Each `tokenId` is a unique blood draw; the `amount` is the number of units from that draw (semi-fungible).

**BloodInfo struct:**

| Field                       | Type    | Values / Purpose                                          |
|-----------------------------|---------|-----------------------------------------------------------|
| <code>bloodGroup</code>     | uint8   | 0=A, 1=B, 2=AB, 3=O                                       |
| <code>rhPositive</code>     | bool    | <code>true</code> =Rh+, <code>false</code> =Rh-           |
| <code>component</code>      | uint8   | 0=Whole, 1=PRBC, 2=Platelets, 3=Plasma                    |
| <code>status</code>         | enum    | Available, Reserved, Sold, Transfused, Expired, Discarded |
| <code>collectionTime</code> | uint64  | Block timestamp at registration                           |
| <code>expiryTime</code>     | uint64  | Future unix timestamp supplied by caller                  |
| <code>donor</code>          | address | Original human donor address                              |
| <code>metadataHash</code>   | bytes32 | keccak256 of full off-chain JSON                          |

**Key functions:**

| Function                               | Access         | Description                                   |
|----------------------------------------|----------------|-----------------------------------------------|
| <code>registerUnit(...)</code>         | MINTER or self | Create token ID, store BloodInfo, mint tokens |
| <code>consumeUnit(tokenId, amt)</code> | Token holder   | Burn tokens after transfusion                 |

| Function                 | Access           | Description                                  |
|--------------------------|------------------|----------------------------------------------|
| flagExpired(tokenId)     | Anyone           | Mark expired; pay 1 YODA bounty to caller    |
| updateStatus(tokenId, s) | Market or holder | Transition lifecycle status                  |
| setMarket(market_)       | Admin            | Link BloodMarket; grant MINTER_ROLE          |
| isExpired(tokenId)       | View             | Returns true if block.timestamp > expiryTime |

**Events:** UnitRegistered, StatusUpdated, ExpiryFlagged, UnitConsumed

### 6.3 BloodDonation

Gateway for altruistic donations. Admin must pre-fund with YODA. Rewards degrade to zero gracefully if balance runs low (no revert).

| Function             | Access     | Description                                     |
|----------------------|------------|-------------------------------------------------|
| donate(...)          | Anyone     | Register donation; tokens → admin; YODA → donor |
| setReward(newReward) | Owner only | Adjust per-donation YODA reward                 |
| withdrawYoda(amount) | Owner only | Reclaim unused YODA from contract               |

**Events:** Donated(tokenId, donor, mintedTo, amount, rewardPaid), RewardUpdated

### 6.4 BloodMarket

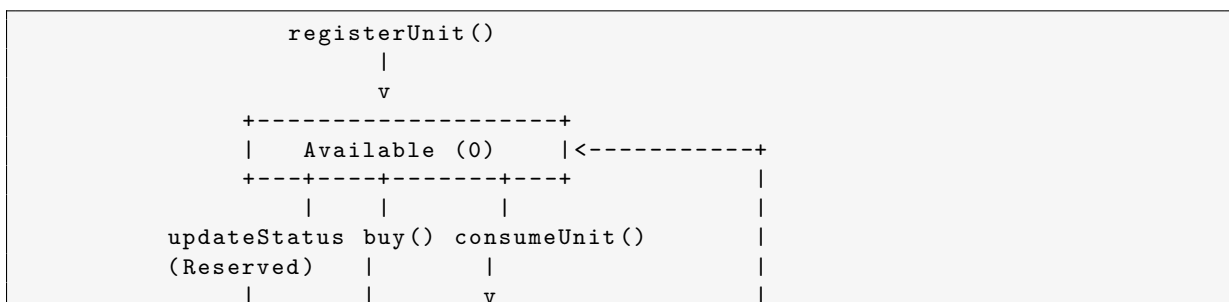
Escrow-based DEX. ReentrancyGuard protects all state-mutating functions.

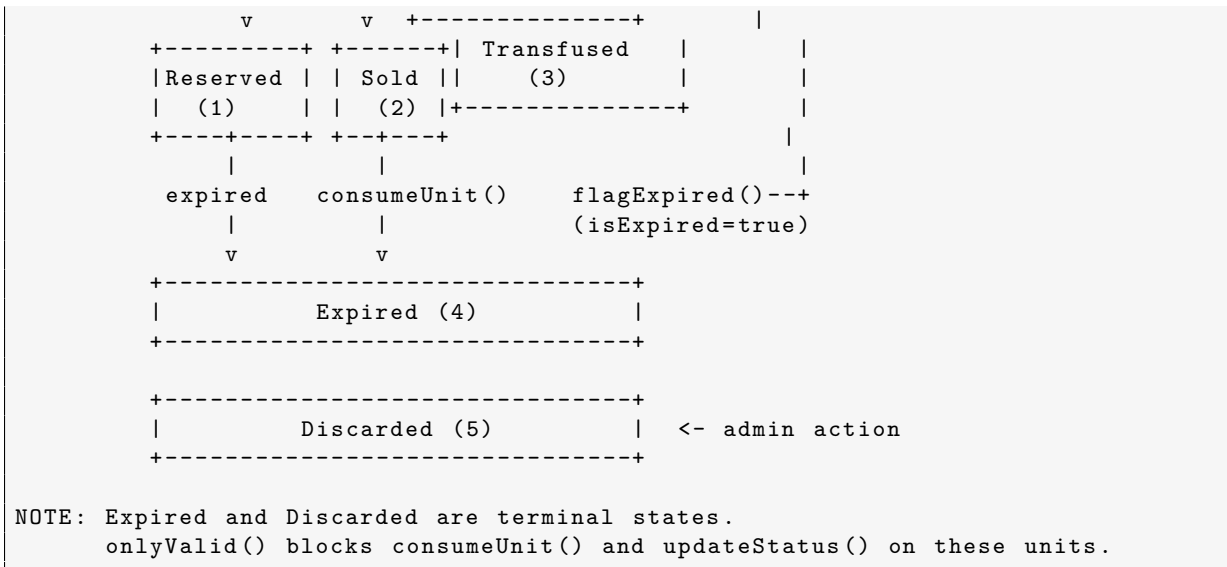
| Function                              | Access         | Description                                    |
|---------------------------------------|----------------|------------------------------------------------|
| createListing(tokenId, amount, price) | Token holder   | Lock tokens in escrow; publish listing         |
| cancelListing(listingId)              | Listing seller | Return tokens to seller; deactivate listing    |
| buy(listingId, amount)                | Anyone         | Route YODA (Path A/B); release tokens to buyer |

**Events:** ListingCreated, ListingFilled, ListingCancelled

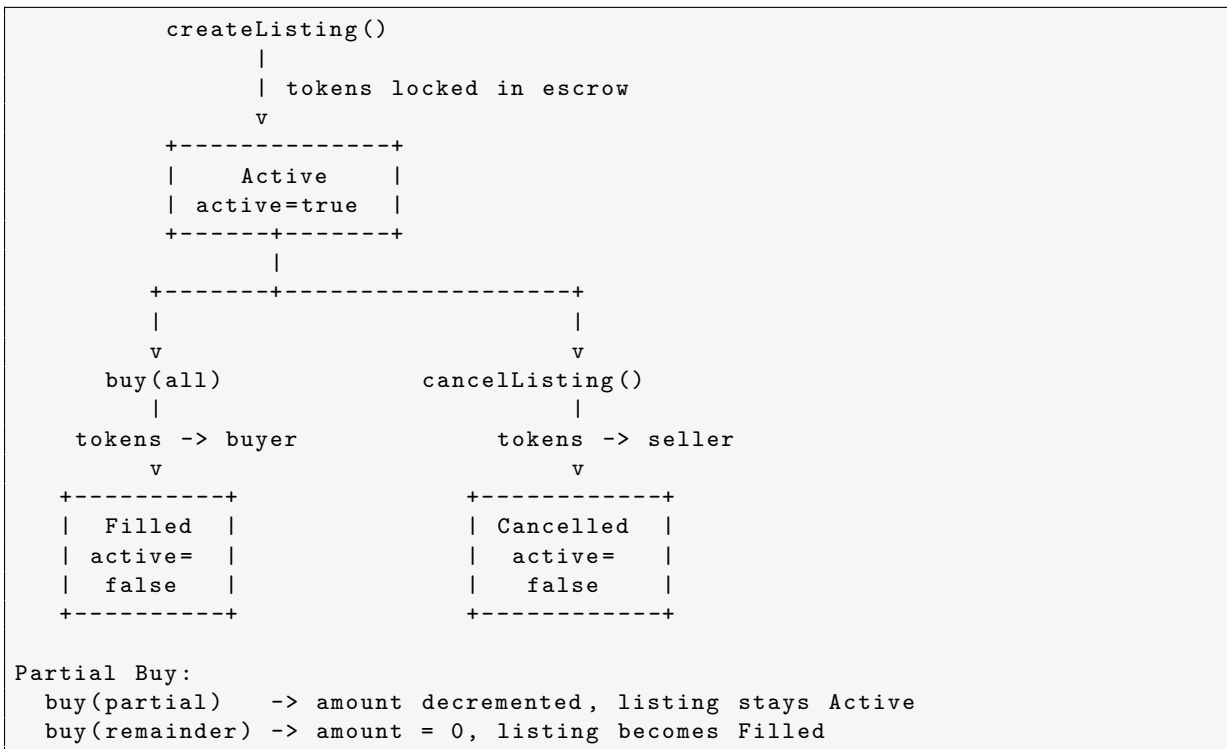
## 7. State Transition Diagrams

### 7.1 Blood Unit Lifecycle





## 7.2 Listing Lifecycle



## 8. Security Analysis

| Threat                | Where         | Mitigation                                   |
|-----------------------|---------------|----------------------------------------------|
| Reentrancy            | BloodMarket   | nonReentrant on all state-mutating functions |
| Double-listing tokens | BloodMarket   | Physical token escrow on createListing()     |
| Unauthorized minting  | BloodRegistry | MINTER_ROLE required for minting to others   |

| Threat                | Where                        | Mitigation                                         |
|-----------------------|------------------------------|----------------------------------------------------|
| Integer overflow      | All                          | Solidity ^0.8.20 built-in overflow protection      |
| ERC-20 silent failure | All                          | <code>SafeERC20.safeTransferFrom</code> throughout |
| Zero-address mistakes | BloodDonation, BloodRegistry | <code>require(addr != address(0))</code> guards    |
| Metadata tampering    | Off-chain                    | keccak256 hash committed on-chain                  |

## 9. Deployment & Wiring

| Step | Action                               | Note                                         |
|------|--------------------------------------|----------------------------------------------|
| 1    | Reference YodaToken                  | External on Sepolia; not redeployed          |
| 2    | Deploy BloodRegistry                 | Pass YODA address + metadata base URI        |
| 3    | Deploy BloodDonation                 | Pass Registry + YODA + reward (500 = 5 YODA) |
| 4    | Deploy BloodMarket                   | Pass YODA + Registry                         |
| 5    | <code>registry.setMarket()</code>    | Grants MINTER_ROLE to BloodMarket            |
| 6    | <code>registry.grantRole(...)</code> | Grants MINTER_ROLE to BloodDonation          |
| 7    | Fund BloodDonation with YODA         | For donor appreciation rewards               |
| 8    | Fund BloodRegistry with YODA         | For expiry bounties                          |

**Key constants:** P2P\_FEE\_BPS = 500 (5% fee), REWARD\_WEI = 500 (5 YODA), EXPIRY\_BOUNTY =  $10^{18}$ .

## 10. Frontend Overview

The frontend is a React + Vite SPA using **ethers.js v6** and MetaMask. ABIs and addresses are centralised in `contracts.js`. YODA amounts use 2 decimal places throughout.

| Page        | Purpose                                                              |
|-------------|----------------------------------------------------------------------|
| Home        | Landing page                                                         |
| Donate      | Altruistic donation form — calls <code>BloodDonation.donate()</code> |
| Dashboard   | View owned tokens; list or flag expired units                        |
| Marketplace | Browse and purchase active blood listings                            |
| Sell        | P2P registration via <code>BloodRegistry.registerUnit()</code>       |